

Introduction

In this experiment, we created a Retrieval Augmented Generation (RAG) system. It answers questions using four main AI policy documents. RAG is a two-step process. First, the system finds relevant text chunks using vector search. Then, it uses a language model to create an answer based on the retrieved text.

I used RecursiveCharacterTextSplitter with a chunk size of 1000 and a chunk overlap of 200 for document preprocessing. This created a balance between keeping context and not cutting off mid-sentence. This is important for policy documents, where arguments often take up several paragraphs. The documents were loaded with PyPDFLoader. They were then split into smaller pieces. These pieces could be encoded and stored in a vector database.

I tested three Sentence Transformer models for embeddings. The models are: all-MiniLM-L6-v2 (baseline), all-mpnet-base-v2 (larger and stronger), and multi-qa-MiniLM-L6-cos-v1 (made for Q&A). The OpenAI gpt-3.5-turbo model used for a generating answers in all runs.

Results & Analysis

Experiment A: Model Comparison-

Table: Average Retrieval and Answer Scores

Model	Avg Retrieval Score	Avg Answer Score
all-MiniLM-L6-v2	3.00	2.40
all-mpnet-base-v2	2.60	2.60
multi-qa-MiniLM-L6-cos-v1	3.00	2.80

➤

EXPERIMENT RESULTS SUMMARY		
Model	Avg Retrieval	Avg Answer
all-MiniLM-L6-v2	3.00	2.40
all-mpnet-base-v2	2.60	2.60
multi-qa-MiniLM-L6-cos-v1	3.00	2.80

Retrieval comparison for Query 1:
all-MiniLM-L6-v2: Retrieval=3, Answer=3
all-mpnet-base-v2: Retrieval=3, Answer=3
multi-qa-MiniLM-L6-cos-v1: Retrieval=3, Answer=3

Retrieval comparison for Query 2:
all-MiniLM-L6-v2: Retrieval=3, Answer=2
all-mpnet-base-v2: Retrieval=2, Answer=2
multi-qa-MiniLM-L6-cos-v1: Retrieval=3, Answer=2

Retrieval comparison for Query 3:
all-MiniLM-L6-v2: Retrieval=3, Answer=3
all-mpnet-base-v2: Retrieval=3, Answer=3
multi-qa-MiniLM-L6-cos-v1: Retrieval=3, Answer=3

Retrieval comparison for Query 4:
all-MiniLM-L6-v2: Retrieval=3, Answer=2
all-mpnet-base-v2: Retrieval=2, Answer=2
multi-qa-MiniLM-L6-cos-v1: Retrieval=3, Answer=3

Retrieval comparison for Query 5:
all-MiniLM-L6-v2: Retrieval=3, Answer=2
all-mpnet-base-v2: Retrieval=3, Answer=3
multi-qa-MiniLM-L6-cos-v1: Retrieval=3, Answer=3

- In this question, we look at how the US and EU define "high-risk AI." The best model took information from both the EU AI Act and the U.S. Executive Order. Meanwhile, the weakest model missed one of these documents entirely, resulting in an incomplete answer. It focused heavily on EU content, which skewed the response.
- This question asked how accountability is explained differently in NIST vs. the AI Bill of Rights. Again, better models managed to retrieve a chunk from each document. The underperforming model missed the AI Bill of Rights entirely, so the generated answer leaned heavily on NIST and gave no contrasting viewpoint.
- Generally, better retrieval led to better answers but not always. In one case (Q1), the retriever returned highly relevant passages, but the generator still gave a vague answer that didn't fully reflect the retrieved content. This shows a generator failure even with good retrieval.

On the other hand, in Q5, even when only one relevant chunk was returned, the answer was clear and accurate indicating a generator success despite weak retrieval.

- The multi-qa-MiniLM-L6-cos-v1 model gave the best overall results, with the highest answer score and perfect retrieval across all queries. Since it is tuned for question-answering, it was better at catching query intent and linking to the right policy content, especially in multi-document and comparative cases. The all-MiniLM-L6-v2 also retrieved well but its

answers were weaker. This shows that the embedding model matters not just for what chunks are pulled but also for how complete and clear the final answers turn out in a RAG setup.

Experiment B: Parameter Tuning

In this parameter tuning experiment using all-MiniLM-L6-v2, I tested different values for chunk size, overlap, and the number of retrieved chunks (k). A chunk size of 500 with moderate overlap (100, 300) provided the best balance between context and retrieval accuracy. When k=2, the system often missed information, while k=6 sometimes pulled extra or less relevant chunks. The setting with k=4 gave the most consistent and contextually rich results. Overall, medium chunking and overlap values led to the most stable and accurate retrieval and answer generation.

Error Analysis

The most difficult question was Q2: How do the U.S. and EU define or handle high-risk AI? The answer failed when the retriever returned content only from the EU document. This was likely a retrieval error due to query terms like “define” and “handle” being too vague to match U.S. text. It could also be a chunking issue where U.S specific content was split or not captured. Using domain specific synonyms or rephrasing the query might improve results in such cases. This issue highlighted how small variations in chunk size and overlap could affect whether the retriever captured the correct parts of the documents, showing how sensitive RAG systems are to preprocessing design.

Conclusion

The key lesson in creating a RAG system with many documents is that retrieval quality limits answer quality. Even a powerful language model can’t generate meaningful answers without the right context. Chunking strategy, embedding model, and even the phrasing of queries all have major impacts on system performance. Balancing these is key to building something truly reliable. The overall experiment showed that multi-qa-MiniLM-L6-cos-v1 is best suited for policy heavy, multi document retrieval tasks because of its strong ability to interpret queries and retrieve context-rich chunks.